UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

INGENIERÍA EN SISTEMAS

AUTÓMATAS

CATEDRATICO: ING. DANILO ANDRÉS ESCOLIN RODRÍGUEZ



Tema: Proyecto Autómatas

Integrantes

Lizbeth Isabel Alvarez Aguilar 3190-24-17694

Daniela Fernanda Gutierrez Franco 3190-24-23395

Fátima Alexandra Monterroso Avendaño 3190-24-16684

Mirella Patzan Alvarez 3190-19-22944

Fátima Ximena Gaitán Guacamaya 3190-24-13220

Guatemala, 17 de octubre de 2026

Índice

Introducción 1

Objetivos 2

Objetivo general 2

Objetivos Específico 2

Descripción del proyecto 3

Arquitectura del software UML 4

Estructuras de datos 6

Desarrollo e implementación 7

Implementación del modelo del autómata 7

Algoritmo de conversión a Regex 9

Descripción general 9

Normalización del autómata 9

Eliminación de estados 9

Obtención y verificación de la expresión regular 10

Manual de usuario 12

1. Introducción 12

2. Inicio del sistema 12

3. Creación del autómata 13

3.1. Agregar estados 13

3.2. Definir los estados inicial y de aceptación 13

3.3. Crear transiciones 13

3.4. Editar y eliminar elementos 14

4. Simulación de cadenas 15

5. Conversión a expresión regular 16

6. Verificación de equivalencia 17

7. Recomendaciones de uso 18

Importación y exportación de autómatas 19

PRUEBAS Y RESULTADOS 20

Cadenas aceptadas y rechazadas 20

Verificación de equivalencia 22
CONCLUSIÓN 24
Bibliografía / egrafía 25
Bibliografía.....25
Egrafía 25
ANEXOS 26

Introducción

En el contexto actual de la informática y las ciencias de la computación, la comprensión de los lenguajes formales y su interacción con la teoría de autómatas resulta fundamental. Este proyecto tiene como finalidad explorar y aplicar conceptos clave relacionados con la equivalencia entre lenguajes regulares, autómatas finitos y expresiones regulares. A través del desarrollo de una herramienta de software, se busca facilitar el aprendizaje de estos conceptos, al tiempo que se promueve una comprensión más profunda de los algoritmos de conversión y manipulación de estos lenguajes.

La tarea de convertir entre autómatas finitos y expresiones regulares es una habilidad esencial en el campo de la computación, con aplicaciones que van desde el análisis de sintaxis en compiladores hasta la búsqueda de patrones en grandes volúmenes de datos. Este proyecto no solo se enfoca en la teoría, sino que también pone en práctica el conocimiento adquirido a través de un enfoque práctico en el desarrollo de software, utilizando tecnologías modernas como JAVA, GITHUB, entre otros que permiten implementar una solución efectiva y eficiente.

A lo largo de este documento, se documentará el proceso de desarrollo, así como las decisiones tomadas y los aprendizajes adquiridos. Este esfuerzo busca contribuir al fortalecimiento del entendimiento teórico y práctico de los temas abordados, tanto para los integrantes del equipo como para cualquier interesado en el área de lenguajes formales y sus aplicaciones

Objetivos

Objetivo general

Desarrollar una aplicación de escritorio en Java que permita crear, editar y simular autómatas finitos deterministas (AFD), así como convertirlos en expresiones regulares mediante el algoritmo de eliminación de estados, utilizando una interfaz gráfica interactiva que facilite la comprensión y aplicación de los conceptos de la teoría de autómatas.

Objetivos Específicos

1. **Implementar una interfaz gráfica interactiva** que permita crear modificar y eliminar estados y transiciones, facilitando la representación visual de autómatas finitos deterministas.
2. **Desarrollar un sistema de simulación de cadenas** que permita visualizar el recorrido de los estados y determinar si una cadena ingresada es aceptada o rechazada por el autómata.
3. **Aplicar el algoritmo de eliminación de estados** para realizar la conversión de autómatas finitos deterministas a expresiones regulares, mostrando visualmente las diferentes etapas del procedimiento.
4. **Implementar un mecanismo de verificación** que permita comparar los resultados obtenidos al evaluar una cadena mediante el autómata y la expresión regular generada.
5. **Organizar el código fuente mediante una arquitectura modular**, utilizando principios de programación orientada a objetos y separación de responsabilidades para facilitar su mantenimiento y comprensión.

Descripción del proyecto

Nuestro proyecto consiste en el desarrollo de una aplicación de escritorio denominada Simulador de Autómatas, diseñada para facilitar la creación, edición, simulación y conversión de autómatas finitos deterministas (AFD) a expresiones regulares. Su propósito principal es proporcionar una herramienta visual e interactiva que permita aplicar los conocimientos adquiridos en el curso de Autómatas y Lenguajes Formales.

El sistema fue desarrollado utilizando el lenguaje de programación Java 17, mediante la biblioteca Java Swing para la construcción de la interfaz gráfica y JGraphX para la representación visual de estados y transiciones. Asimismo, se utilizó Apache Maven para la gestión de dependencias y organización del proyecto.

La aplicación cuenta con un lienzo interactivo en el que el usuario puede agregar estados, establecer transiciones, definir un estado inicial e identificar estados de aceptación. También permite modificar y eliminar elementos del autómata, facilitando la construcción de diferentes estructuras.

Una de sus principales funcionalidades es la simulación de cadenas, mediante la cual el usuario puede ingresar una secuencia de símbolos y observar el recorrido realizado por el autómata. Al finalizar el proceso, el sistema determina si la cadena es aceptada o rechazada según los estados y transiciones establecidos.

Otra funcionalidad importante es la conversión de autómatas finitos a expresiones regulares, utilizando el algoritmo de eliminación de estados. Este procedimiento consiste en normalizar el autómata, eliminar progresivamente los estados intermedios y actualizar las transiciones hasta obtener una expresión regular que represente el lenguaje reconocido por el autómata original.

Adicionalmente, el programa incorpora un mecanismo de verificación de equivalencia para cadenas de prueba, que permite comparar los resultados de la simulación del autómata con la evaluación de la expresión regular obtenida.

Para su desarrollo, el código fuente se organizó en diferentes paquetes, separando las clases encargadas de representar los elementos matemáticos, controlar las operaciones, ejecutar los algoritmos y gestionar la interfaz gráfica.

En conjunto, el proyecto integra conceptos teóricos de lenguajes formales con herramientas de programación orientada a objetos, ofreciendo una solución práctica para visualizar y comprender el funcionamiento de los autómatas finitos y su relación con las expresiones regulares.

Arquitectura del software UML

El sistema se desarrolló utilizando una arquitectura modular basada en los principios de la programación orientada a objetos (POO), organizando sus funcionalidades en diferentes paquetes según sus responsabilidades. Esta estructura permite separar la representación de los autómatas, el control de operaciones, la simulación de cadenas, la conversión a expresiones regulares, la gestión de archivos y la interfaz gráfica.

El siguiente diagrama UML presenta una vista general de las clases principales del proyecto y las relaciones entre ellas, facilitando la comprensión de la organización y funcionamiento del sistema.

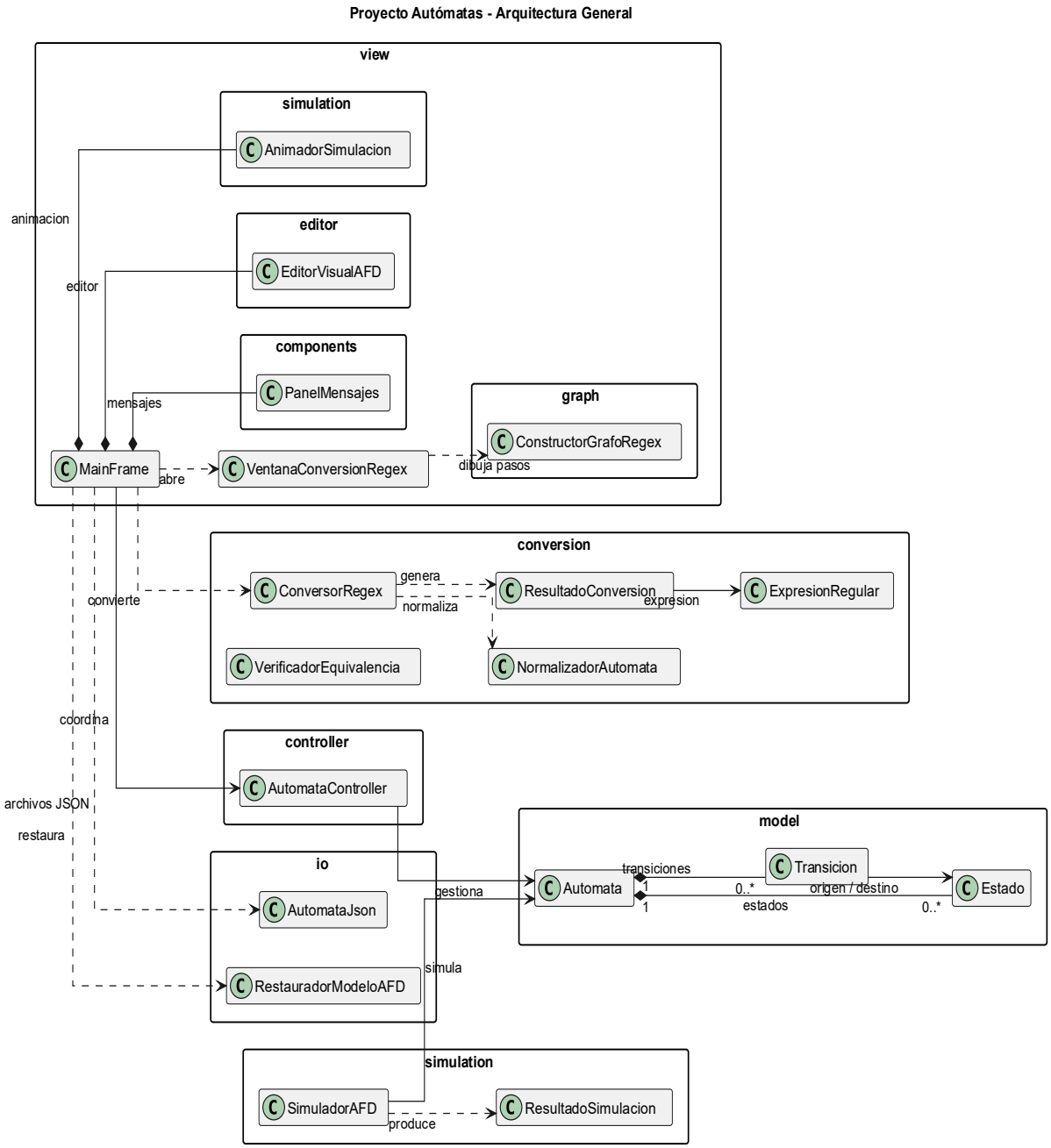


Figura 1. Diagrama UML general de la arquitectura del sistema de autómatas.

Para facilitar su lectura, este diagrama presenta únicamente las clases y relaciones principales del sistema. Los diagramas UML detallados, que incluyen atributos, métodos y componentes adicionales, se encuentran en la sección de anexos.

Estructuras de datos

El sistema utiliza diferentes estructuras de datos para almacenar, organizar y manipular los componentes de los autómatas finitos deterministas (AFD). Estas estructuras permiten administrar los estados, las transiciones y los símbolos del alfabeto, además de facilitar la simulación de cadenas y la conversión a expresiones regulares.

Estados: Se representan mediante la clase Estado, que almacena información como el nombre, las coordenadas y las propiedades que identifican si un estado es inicial o de aceptación. Los estados del autómata se almacenan en una lista de tipo ArrayList, mientras que los estados de aceptación se administran mediante un conjunto de tipo HashSet.

Transiciones: La clase Transicion representa las conexiones dirigidas entre estados. Cada transición contiene un estado de origen, un estado de destino y un símbolo de tipo String. Estas transiciones se almacenan en una lista que permite recorrerlas durante la simulación y conversión del autómata.

Alfabeto: Los símbolos utilizados por el autómata se administran mediante un conjunto de tipo HashSet, evitando la duplicación de elementos y facilitando la validación de las cadenas ingresadas.

Grafos: El autómata se representa matemáticamente como un grafo dirigido G = (Q, E), donde Q corresponde al conjunto de estados y E al conjunto de transiciones. Para su representación gráfica se utiliza la biblioteca JGraphX, que permite visualizar los estados como nodos y las transiciones como conexiones dirigidas.

Mapas: Se utilizan estructuras de tipo Map para relacionar los elementos visuales de JGraphX con los objetos del modelo del autómata. Esto facilita la identificación, actualización y manipulación de los estados desde la interfaz gráfica.

Conversión a expresiones regulares: La clase ConversorRegex utiliza las estructuras del autómata y la información del grafo normalizado para aplicar el algoritmo de eliminación de estados. Durante este procedimiento se actualizan las expresiones regulares asociadas a las transiciones, conservando los caminos que permiten reconocer el mismo lenguaje.

Estas estructuras contribuyen a mantener una organización modular del sistema y permiten ejecutar correctamente sus principales funcionalidades: edición de autómatas, simulación de cadenas, conversión a expresiones regulares y verificación de resultados.

Desarrollo e implementación

El sistema se desarrolló utilizando el lenguaje de programación Java y el entorno de desarrollo Apache NetBeans. Para la construcción de la interfaz gráfica se utilizó Java Swing, mientras que la biblioteca JGraphX permitió representar visualmente los estados y las transiciones de los autómatas.

La implementación se organizó en diferentes paquetes, aplicando principios de programación orientada a objetos para separar la representación matemática de los autómatas, el procesamiento de los algoritmos y los componentes de la interfaz gráfica.

Implementación del modelo del autómata

Para representar los componentes de un autómata finito determinista (AFD), se implementaron las clases Automata, Estado y Transicion, ubicadas en el paquete model.

La clase Automata se encarga de almacenar los estados, el alfabeto, las transiciones, el estado inicial y los estados de aceptación. Para organizar estos elementos se utilizaron estructuras como ArrayList y HashSet, que permiten administrar los datos necesarios para el funcionamiento del sistema.

Fragmento de código de la clase Automata.java:

```
public class Automata {  aleja +1

    private final List<Estado> estados;  10 usages
    private final Set<String> alfabeto;  5 usages
    private final List<Transicion> transiciones;  8 usages
    private Estado estadoInicial;  9 usages
    private final Set<Estado> estadosAceptacion;  6 usages

    public Automata() {  Lizbeth +1
        estados = new ArrayList<>();
        alfabeto = new HashSet<>();
        transiciones = new ArrayList<>();
        estadosAceptacion = new HashSet<>();
        estadoInicial = null;
    }
```

Figura 2. Declaración e inicialización de las estructuras de datos de la clase Automata.

En el fragmento de código se observa que los estados y las transiciones se almacenan mediante listas de tipo ArrayList, mientras que el alfabeto y los estados de aceptación utilizan conjuntos de tipo HashSet. Asimismo, el constructor inicializa estas estructuras y establece el estado inicial con un valor `null`, hasta que el usuario seleccione uno desde la interfaz gráfica.

Algoritmo de conversión a Regex

Descripción general

El sistema implementa el algoritmo de eliminación de estados para convertir un autómata finito determinista (AFD) en una expresión regular equivalente. Este procedimiento consiste en transformar progresivamente el grafo del autómata, combinando las expresiones asociadas a sus transiciones hasta obtener una expresión regular que represente el lenguaje reconocido.

Normalización del autómata

El proceso inicia con la normalización del autómata, mediante la incorporación de dos estados auxiliares: un nuevo estado inicial, denominado “q_start”, y un nuevo estado de aceptación, denominado “q_accept”.

El estado q_start se conecta con el estado inicial original mediante una transición épsilon (ε), mientras que los estados de aceptación originales se conectan con q_accept utilizando transiciones épsilon.

Esta transformación permite disponer de un único estado de entrada y un único estado de salida, facilitando la aplicación del algoritmo sin modificar el lenguaje reconocido por el autómata.

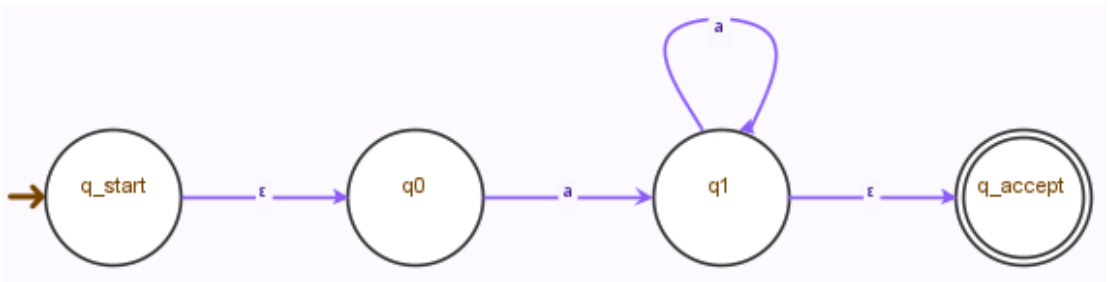


Figura 1. Autómata normalizado con los estados auxiliares q_start y q_accept.

Eliminación de estados

Una vez normalizado el autómata, se eliminan progresivamente los estados intermedios. Durante este procedimiento, las expresiones regulares de las transiciones se actualizan para conservar los caminos que existían a través del estado eliminado.

Para realizar esta actualización se utiliza una fórmula que combina las transiciones existentes con los caminos que atraviesan el estado que se elimina.

$$R'_{\{ik\}} = R_{\{ik\}} \cup R_{\{ij\}} \cdot \left(R_{\{jj\}}\right)^* \cdot R_{\{jk\}}$$

En esta fórmula:

- R(i,k) representa la expresión regular de la transición existente desde el estado i hasta el estado k.
- R(i,j) representa la expresión que conecta el estado i con el estado j.
- R(j,j) representa los ciclos del estado j, sobre los cuales se aplica la cerradura de Kleene (*).
- R(j,k) representa la expresión que conecta el estado j con el estado k.

La operación combina unión, concatenación y cerradura de Kleene para mantener los caminos posibles del autómata. El procedimiento continúa hasta conservar únicamente los estados auxiliares inicial y final.

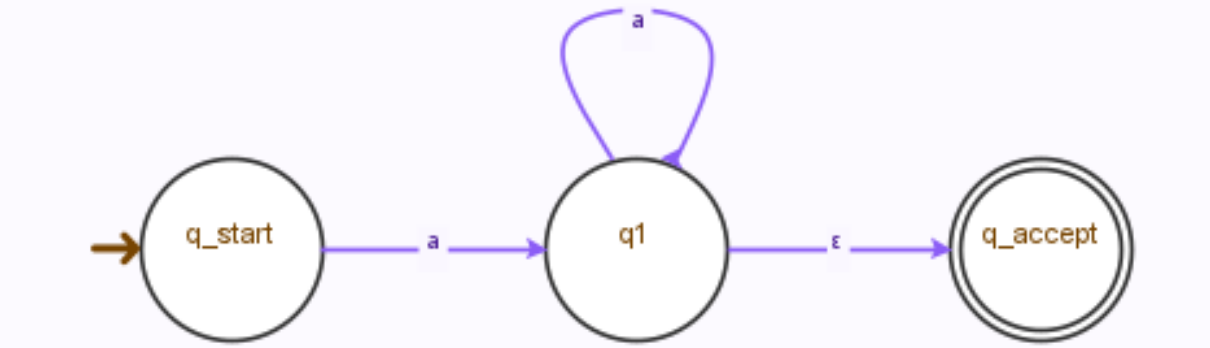


Figura 2. Eliminación de estados intermedios durante el proceso de conversión.

Obtención y verificación de la expresión regular

Al finalizar la eliminación de estados, la expresión asociada al camino entre q_start y q_accept constituye la expresión regular resultante.

Posteriormente, la clase VerificadorEquivalencia permite comparar la aceptación de cadenas de prueba mediante el AFD y la expresión regular generada, utilizando el motor de expresiones regulares de Java.

Cuando ambos métodos aceptan o rechazan una misma cadena, sus resultados coinciden para el caso evaluado. Esta comprobación permite identificar posibles discrepancias y evaluar el funcionamiento de la conversión.

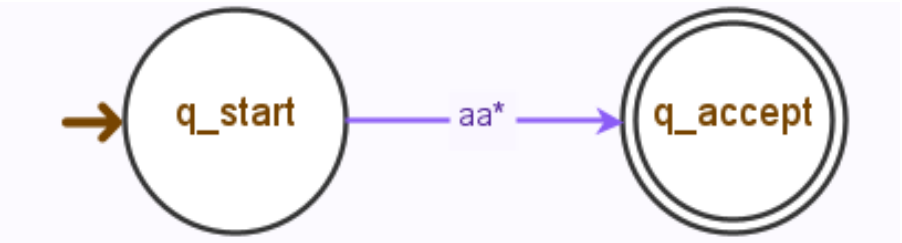


Figura 3. Expresión regular final obtenida a partir del autómata.

En el ejemplo mostrado, la expresión regular obtenida fue aa*, la cual representa una o más ocurrencias del símbolo a.

La siguiente tabla presenta las posibles interpretaciones de dicha comparación.

Resultado del AFD	**Resultado de Regex**	**Interpretación**
Aceptada	Aceptada	Coinciden los resultados
Rechazada	Rechazada	Coinciden los resultados
Aceptada	Rechazada	Existe una discrepancia
Rechazada	Aceptada	Existe una discrepancia

Tabla 1. Interpretación de los resultados de verificación entre el AFD y la expresión regular.

Manual de usuario

1. Introducción

El presente manual tiene como finalidad orientar al usuario en el manejo del sistema Simulador de Autómatas, desarrollado en Java. La aplicación permite crear, modificar y simular autómatas finitos deterministas (AFD), así como convertirlos a expresiones regulares mediante el algoritmo de eliminación de estados.

A continuación, se describen los procedimientos necesarios para utilizar las principales funcionalidades del programa.

2. Inicio del sistema

Para iniciar la aplicación, se debe abrir el proyecto en Apache NetBeans y ejecutarlo mediante la opción Run Project (Ejecutar proyecto).

Una vez iniciado, se mostrará la ventana principal del sistema, que contiene las siguientes áreas:

- Panel de herramientas: permite agregar, modificar y eliminar estados y transiciones, además de importar y exportar autómatas.
- Lienzo del autómata: espacio donde se construye y visualiza el AFD.
- Panel de simulación: permite ingresar cadenas y comprobar si son aceptadas o rechazadas.
- Mensajes del sistema: muestra indicaciones sobre las operaciones realizadas.

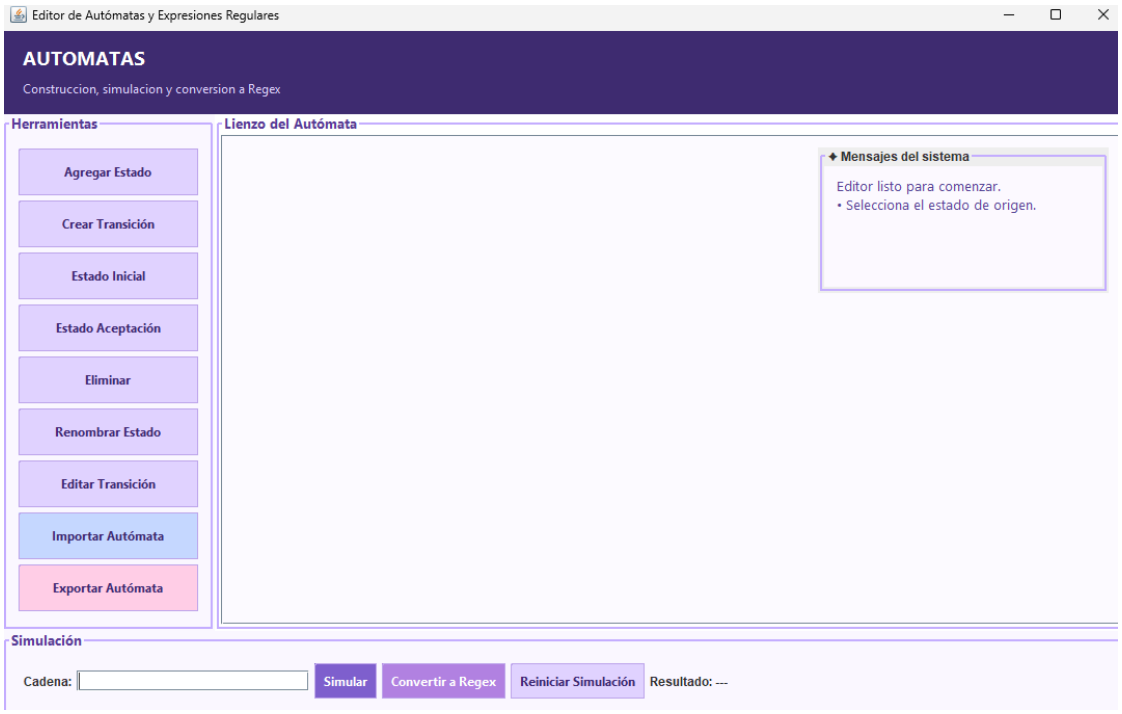


Figura. Interfaz principal del simulador de autómatas.

3. Creación del autómata

Para construir un autómata, el usuario debe crear sus estados, establecer el estado inicial, identificar los estados de aceptación y agregar las transiciones correspondientes.

3.1. Agregar estados

1. Presionar el botón Agregar Estado, ubicado en el panel de herramientas.
2. Hacer clic sobre el lienzo en la posición donde se desea colocar el estado.
3. Repetir el procedimiento para crear los estados adicionales.

El sistema asigna automáticamente nombres como q0, q1, q2, según el orden de creación.

3.2. Definir los estados inicial y de aceptación

1. Seleccionar el estado que será el inicial.
2. Presionar Estado Inicial. El sistema lo identificará mediante una flecha de entrada.
3. Seleccionar el estado que será de aceptación.
4. Presionar Estado Aceptación. Este se identificará mediante un doble círculo.

El usuario puede modificar posteriormente estas propiedades seleccionando los estados correspondientes.

3.3. Crear transiciones

1. Presionar el botón Crear Transición.
2. Seleccionar el estado de origen.
3. Seleccionar el estado de destino.
4. En la ventana emergente, introducir el símbolo correspondiente.
5. Confirmar la operación.

Para crear un bucle, se debe seleccionar el mismo estado como origen y destino.

Ejemplo práctico: se puede crear un autómata con los estados q0 y q1, donde q0 es inicial y q1 es de aceptación. Se agrega una transición de q0 hacia q1 con el símbolo a y un bucle en q1 con el mismo símbolo.

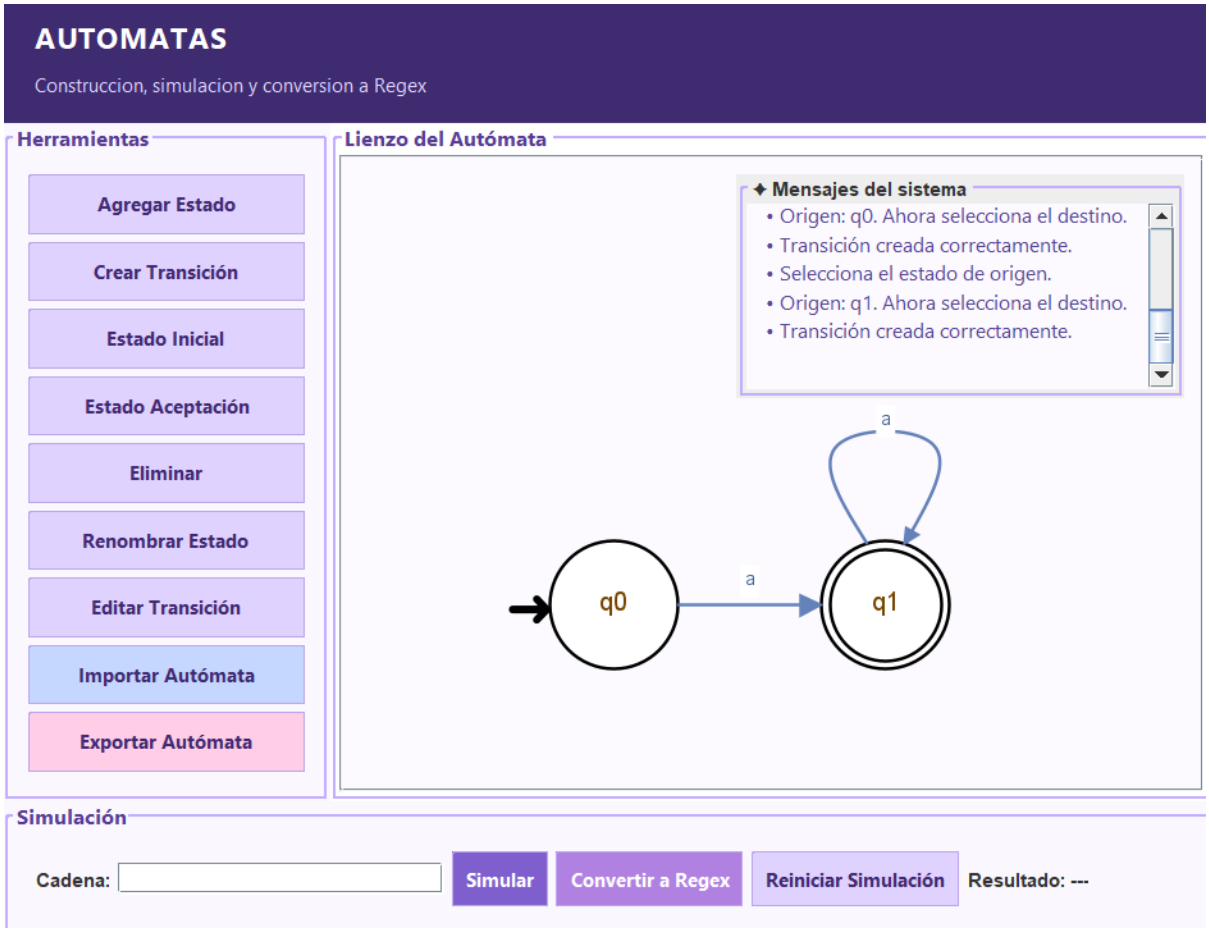


Figura. Construcción de un AFD mediante estados y transiciones.

3.4. Editar y eliminar elementos

El programa también permite realizar modificaciones sobre el autómata construido.

- **Renombrar Estado:** seleccionar un estado, presionar el botón correspondiente e ingresar el nuevo nombre.
- Editar Transición: seleccionar una transición, presionar Editar Transición e introducir el nuevo símbolo.
- **Eliminar:** seleccionar un estado o transición y presionar Eliminar. Cuando se elimina un estado, el sistema solicita confirmación.
- **Mover estados:** arrastrar los estados dentro del lienzo para modificar su posición.

Estas herramientas permiten corregir y reorganizar el autómata sin necesidad de crearlo nuevamente.

4. Simulación de cadenas

La simulación permite comprobar si una cadena pertenece al lenguaje reconocido por el autómata construido.

Para realizarla:

1. Verificar que el autómata tenga un estado inicial y las transiciones necesarias.
2. Localizar el campo Cadena, ubicado en la parte inferior de la ventana principal.
3. Escribir la cadena que se desea evaluar.
4. Presionar el botón Simular.
5. Observar el recorrido resaltado de los estados y las transiciones.
6. Revisar el resultado presentado por el sistema.

Al finalizar, la aplicación indica si la cadena fue ACEPTADA o RECHAZADA.

Por ejemplo, utilizando el autómata del apartado anterior, la cadena aaa será aceptada, porque permite llegar al estado q1 y permanecer en él mediante su bucle. En cambio, una cadena vacía será rechazada porque el estado inicial q0 no es de aceptación.

Para realizar otra prueba, se puede presionar Reiniciar Simulación, lo que limpia la entrada, el resultado anterior y los colores utilizados durante el recorrido.

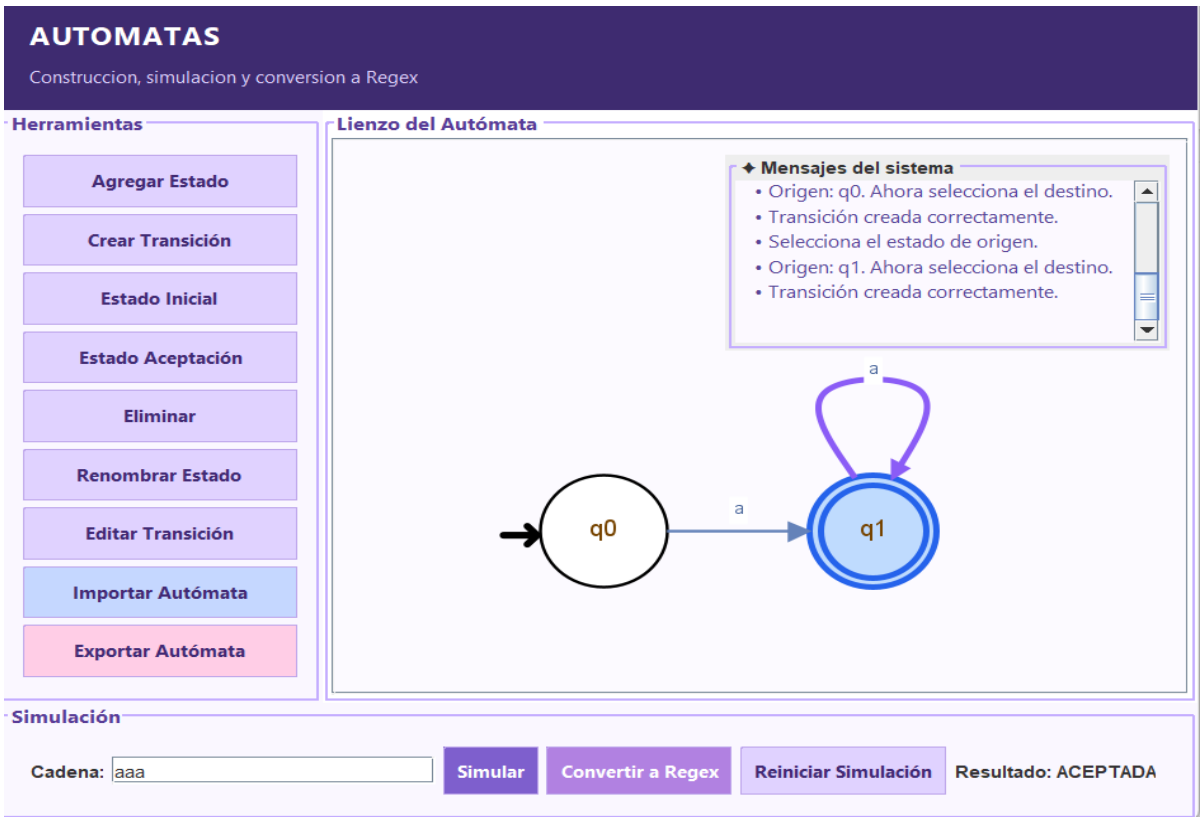


Figura. Simulación de una cadena aceptada por el autómata.

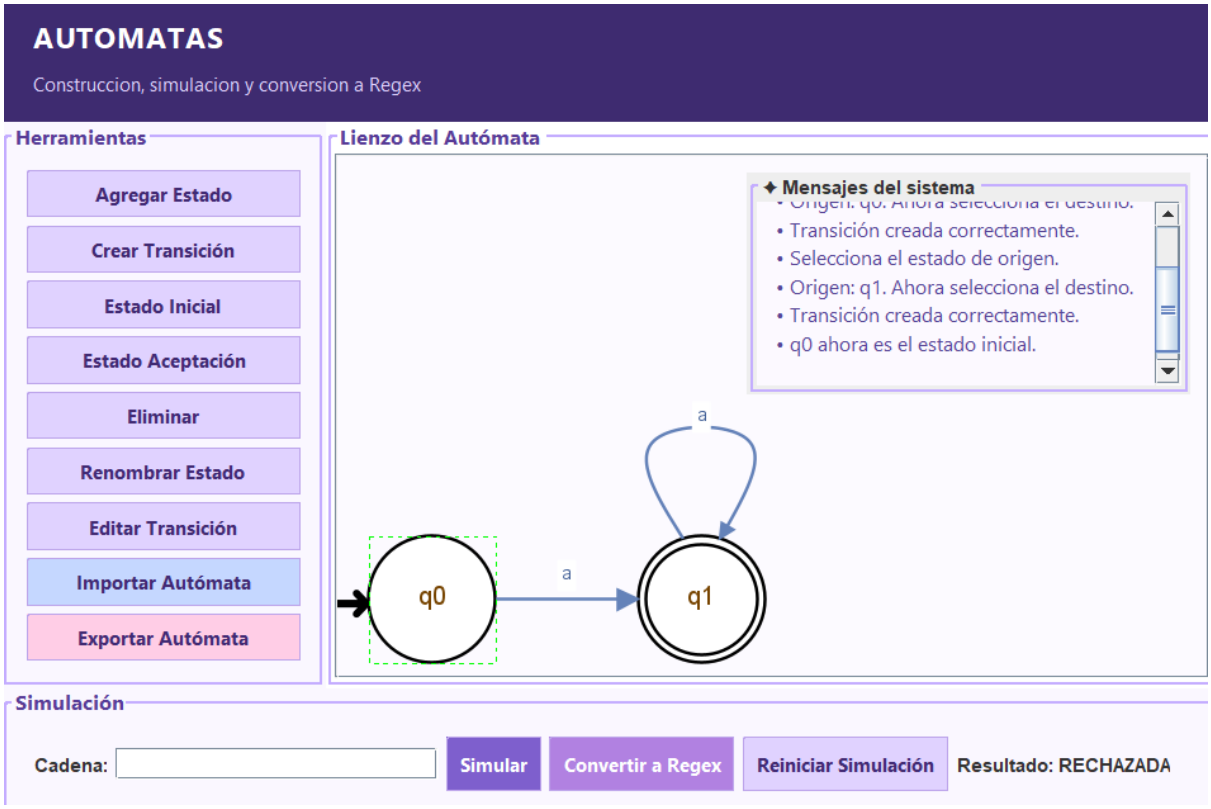


Figura. Simulación de una cadena vacía rechazada por el autómata.

5. Conversión a expresión regular

El sistema permite convertir el autómata construido en una expresión regular utilizando el algoritmo de eliminación de estados.

Para iniciar el procedimiento:

1. Construir un autómata con sus estados y transiciones.
2. Presionar el botón Convertir a Regex en la ventana principal.
3. Se abrirá la ventana Conversión de autómata a Regex, mostrando una copia del autómata original.
4. Presionar Iniciar conversión.
5. Utilizar Siguiente y Anterior para observar las distintas etapas del procedimiento.
6. También se puede seleccionar Reproducir para visualizar automáticamente los pasos, o Pausar para detener la reproducción.

Durante el proceso, se muestra la normalización del autómata, la eliminación progresiva de estados y las actualizaciones realizadas en las transiciones.

Al completar los pasos, el sistema presenta la expresión obtenida en el campo Regex final. La ventana también dispone de controles de zoom para ampliar o reducir la visualización del grafo.

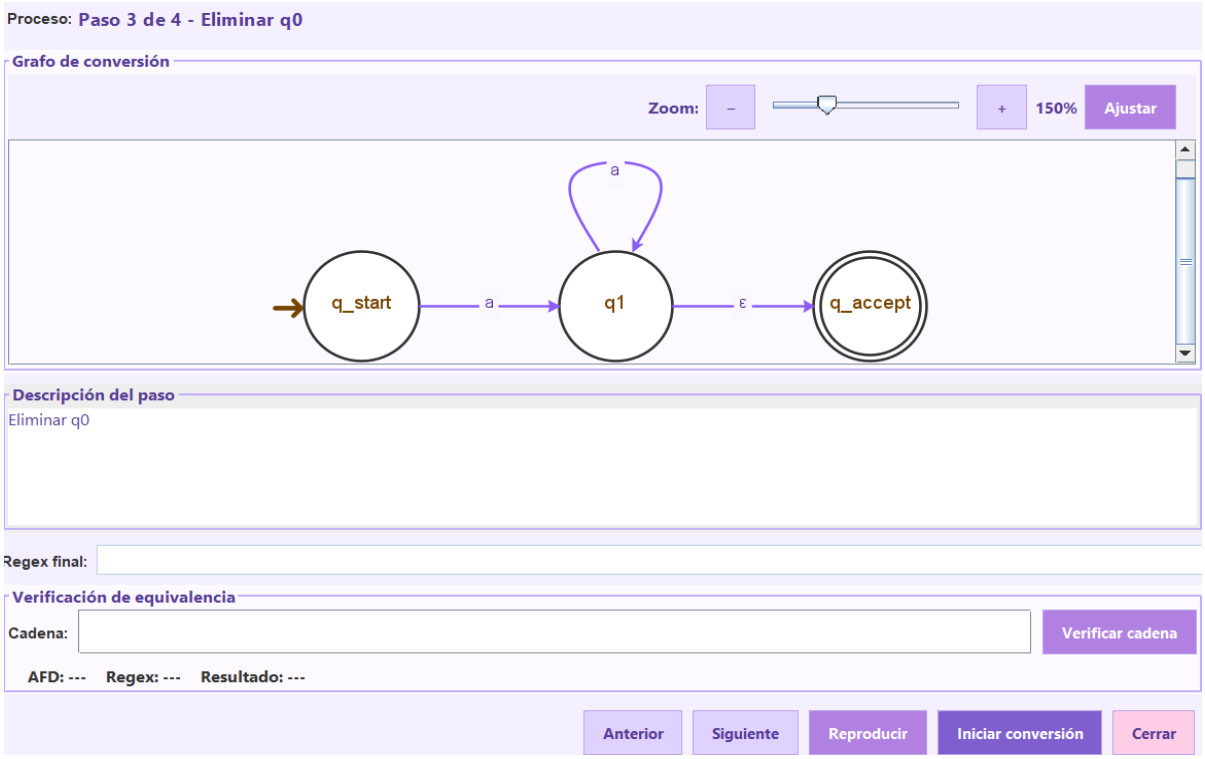


Figura. Visualización del proceso de eliminación de estados.

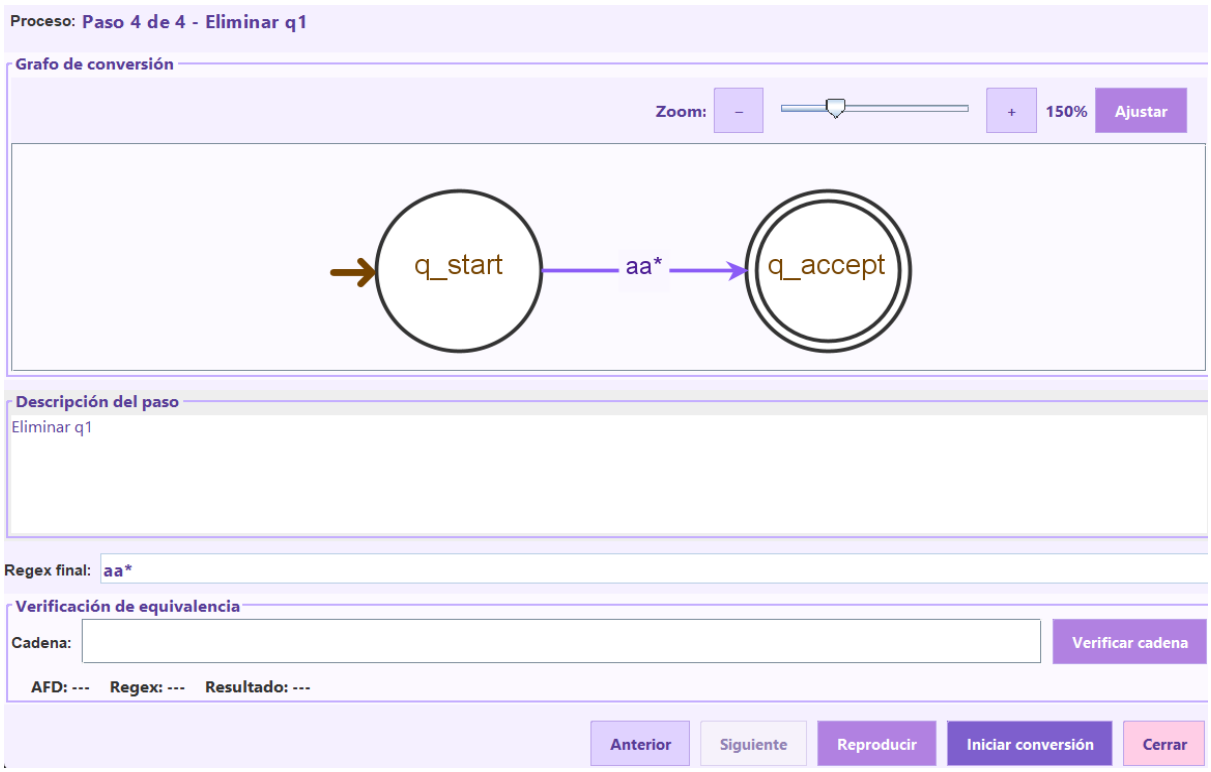


Figura. Expresión regular obtenida mediante el algoritmo de conversión.

6. Verificación de equivalencia

Después de realizar la conversión, el programa permite comprobar si el autómata y la expresión regular generada producen los mismos resultados para una cadena de prueba.

El procedimiento es el siguiente:

1. Completar la conversión del autómata a expresión regular.
2. Localizar el apartado Verificación de equivalencia en la ventana de conversión.
3. Introducir una cadena en el campo Cadena.
4. Presionar Verificar cadena.
5. Observar los resultados de aceptación o rechazo correspondientes al AFD y a la expresión regular.

El sistema muestra tres indicadores:

- **AFD:** resultado obtenido mediante el autómata.
- **Regex:** resultado obtenido mediante la expresión regular.
- **Resultado:** indica si ambas evaluaciones coinciden.

Cuando los dos métodos aceptan o rechazan la misma cadena, el sistema presenta el mensaje Coinciden. Si producen respuestas diferentes, muestra No coinciden.

Esta función permite comprobar el comportamiento de ambos métodos con las cadenas evaluadas.

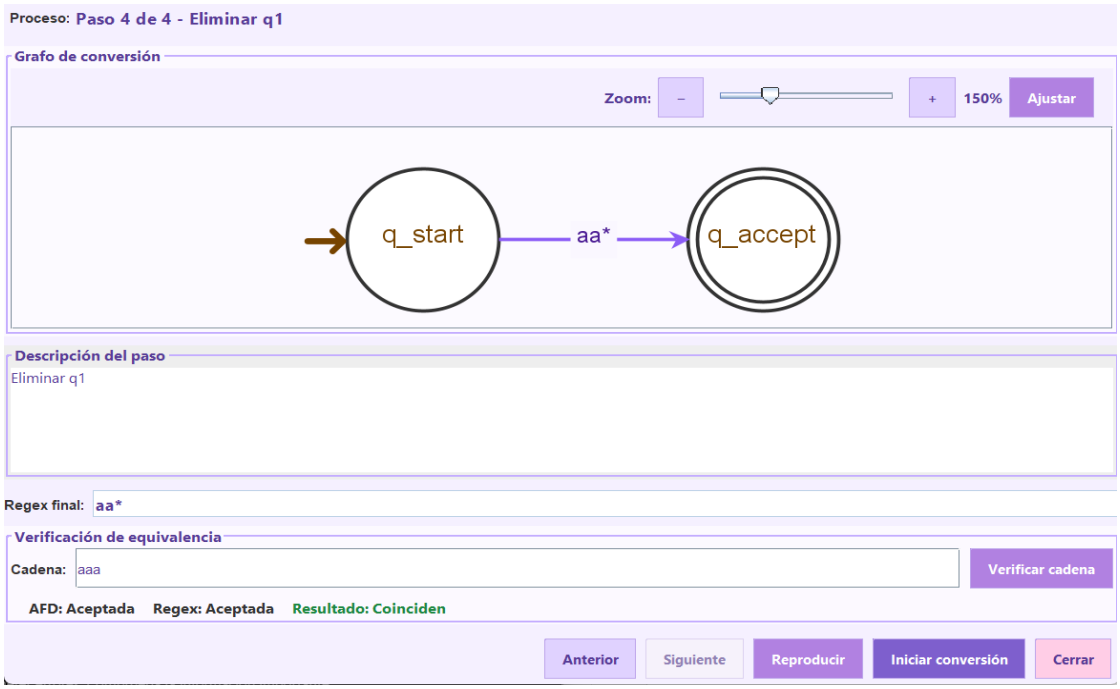


Figura. Verificación de resultados entre el autómata y la expresión regular.

7. Recomendaciones de uso

Para utilizar correctamente el sistema, se recomienda:

- Definir el estado inicial y los estados de aceptación antes de realizar las simulaciones.
- Verificar que cada transición esté conectada con los estados correspondientes.
- Utilizar los símbolos definidos en el alfabeto del autómata.
- Emplear Reiniciar Simulación antes de realizar una nueva prueba cuando se desee limpiar el recorrido anterior.
- Revisar los diferentes pasos de la conversión para comprender el procedimiento de eliminación de estados.
- Probar cadenas aceptadas y rechazadas para evaluar el funcionamiento del autómata.

Importación y exportación de autómatas

Adicionalmente, el sistema permite guardar y recuperar autómatas mediante archivos JSON.

Para guardar un autómata, presionar Exportar Autómata, seleccionar la ubicación del archivo y confirmar la operación.

Para recuperar uno previamente guardado, presionar Importar Autómata, seleccionar el archivo JSON correspondiente y abrirlo. Si existe un autómata en el lienzo, el programa solicitará confirmación antes de reemplazarlo.

Estas funciones permiten conservar el trabajo realizado y continuar modificándolo posteriormente.

PRUEBAS Y RESULTADOS

Cadenas aceptadas y rechazadas

Para comprobar el funcionamiento del sistema, se realizaron pruebas utilizando un autómata finito determinista (AFD) compuesto por tres estados: q0, q1 y q2. El estado q0 se estableció como inicial y q2 como estado de aceptación.

El autómata utiliza el alfabeto {a, b} y contiene seis transiciones que permiten reconocer cadenas cuyo contenido termina con la secuencia ab.

Se evaluaron diferentes cadenas de entrada para comprobar que el simulador identifica correctamente las condiciones de aceptación y rechazo.

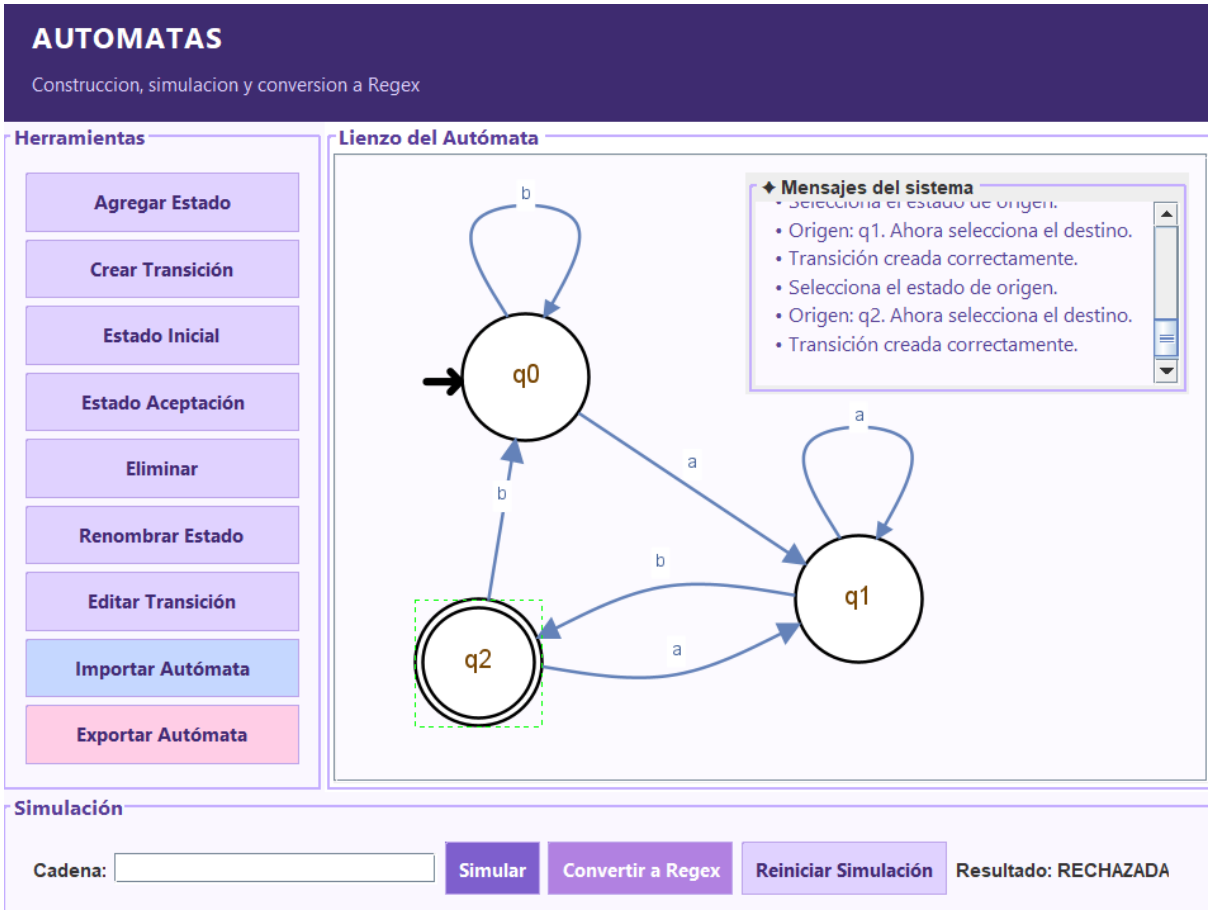


Figura. Autómata de tres estados utilizado para realizar las pruebas.

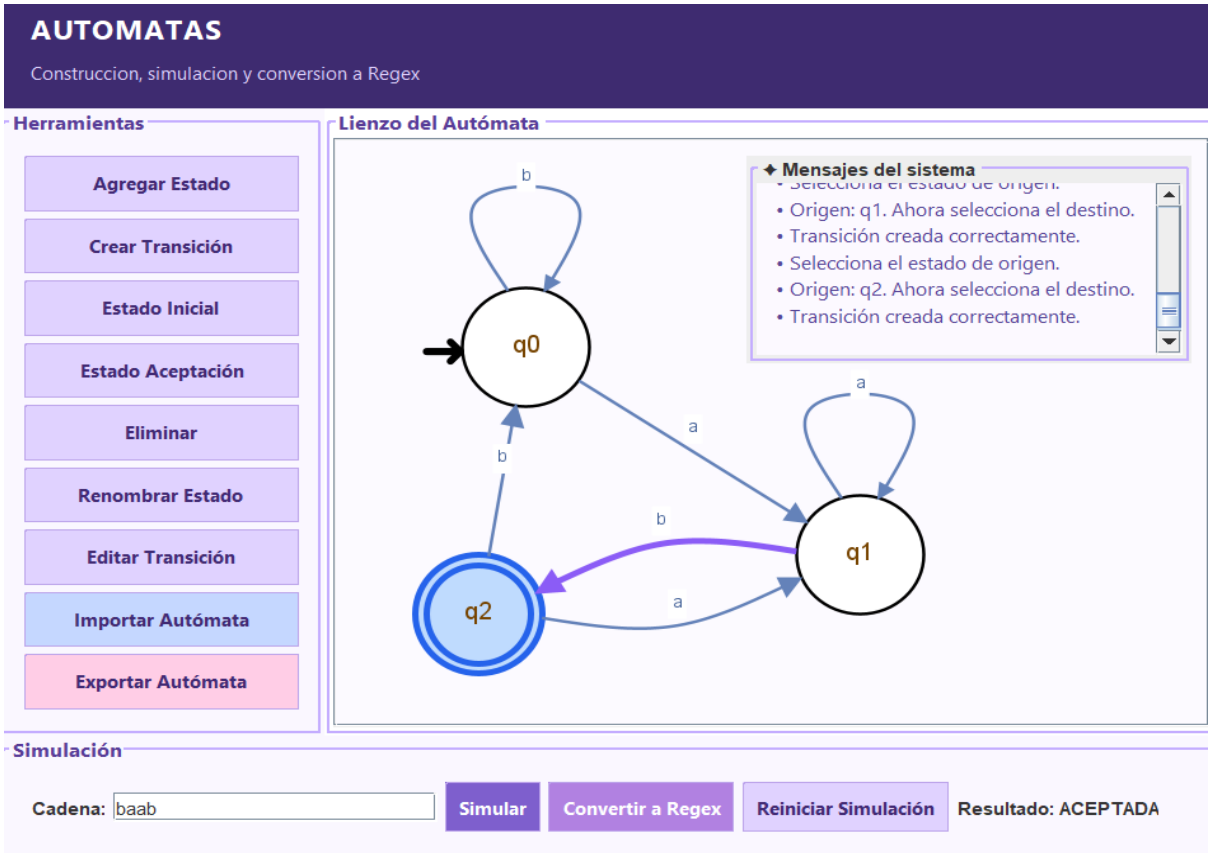


Figura. Simulación de una cadena aceptada por el autómata.

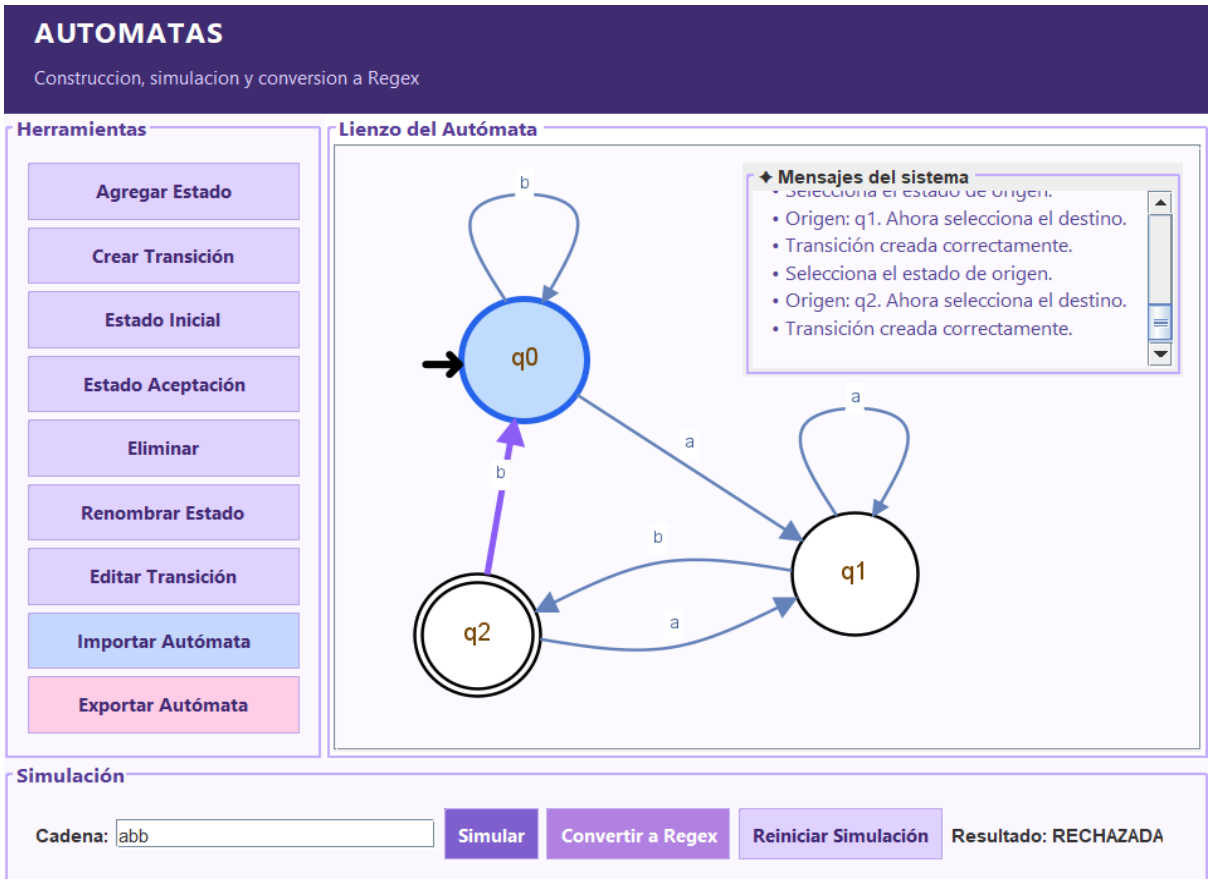


Figura. Simulación de una cadena rechazada por el autómata.

Verificación de equivalencia

Posteriormente, se realizó la conversión del autómata a una expresión regular mediante el algoritmo de eliminación de estados.

La expresión regular generada por el sistema fue: b*aa*b((a | bb*a)a*b)*

Esta expresión es equivalente a la forma simplificada (a | b) * a b, que representa las cadenas formadas por los símbolos a y b que terminan en la secuencia ab.

Aunque la expresión obtenida es más extensa, esto se debe a las operaciones de unión, concatenación y cerradura de Kleene aplicadas durante el proceso de eliminación de estados.

Para verificar su funcionamiento, se compararon los resultados de aceptación y rechazo del autómata con los obtenidos mediante la expresión regular generada.

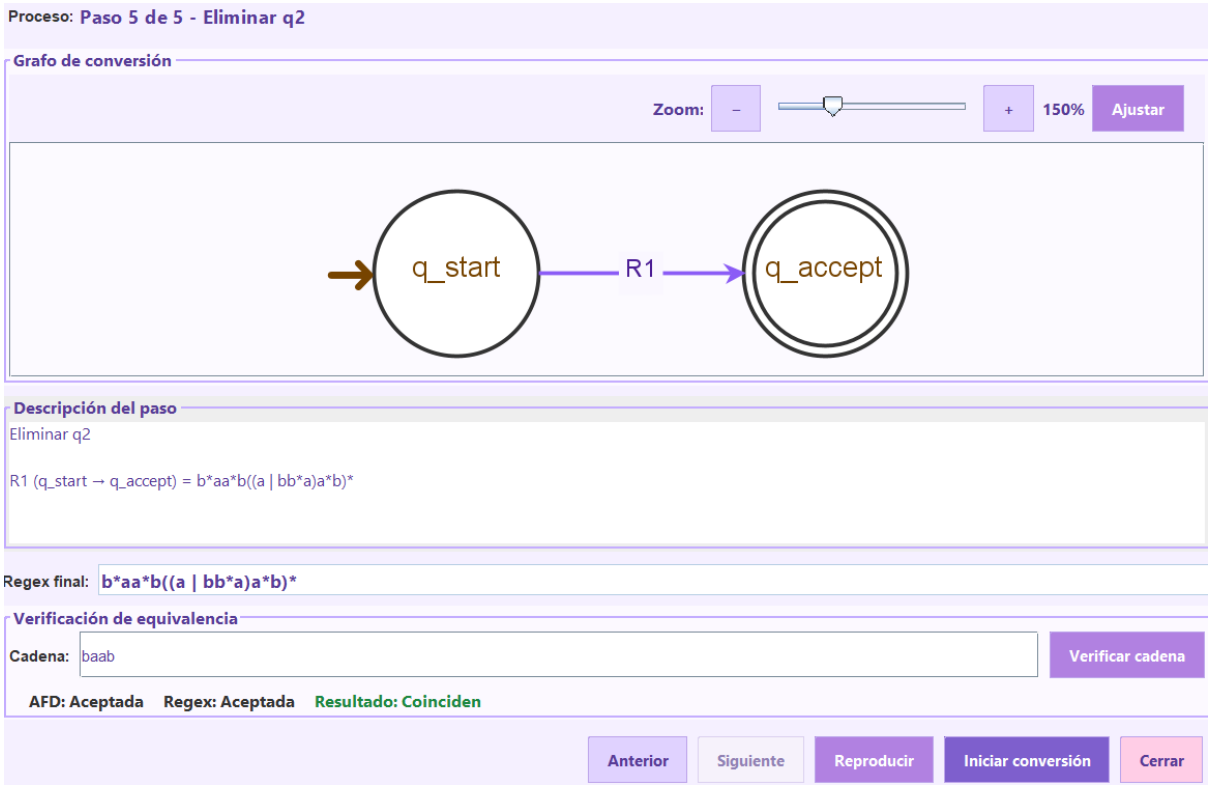


Figura. Verificación de equivalencia para una cadena aceptada.

Resultados

Las pruebas permitieron evaluar el funcionamiento de la simulación y la conversión a expresiones regulares. Se utilizaron cadenas aceptadas, rechazadas y la cadena vacía para comprobar diferentes situaciones.

Cadena	Resultado AFD	Resultado Regex	Comparación
ab	Aceptada	Aceptada	Coinciden
aab	Aceptada	Aceptada	Coinciden
bab	Aceptada	Aceptada	Coinciden
baab	Aceptada	Aceptada	Coinciden
abb	Rechazada	Rechazada	Coinciden
aba	Rechazada	Rechazada	Coinciden
bbb	Rechazada	Rechazada	Coinciden
Cadena vacía (ε)	Rechazada	Rechazada	Coinciden

Tabla. Resultados de las pruebas realizadas con el autómata de tres estados.

Los resultados obtenidos permiten comprobar si las cadenas evaluadas cumplen con la condición establecida por el autómata. Asimismo, la comparación con la expresión regular generada permite verificar que ambos procedimientos presentan el mismo comportamiento en los casos analizados.

Estas pruebas constituyen evidencia del funcionamiento de la simulación, la conversión a expresiones regulares y el mecanismo de verificación implementado en el sistema.

CONCLUSIÓN

El desarrollo del proyecto permitió aplicar de manera práctica los conocimientos adquiridos sobre autómatas finitos deterministas (AFD) y expresiones regulares, mediante la implementación de una aplicación de escritorio en Java. La utilización de una interfaz gráfica interactiva facilitó la creación, edición y visualización de autómatas, así como la simulación de cadenas para determinar su aceptación o rechazo.

La implementación del algoritmo de eliminación de estados permitió comprender la relación entre los autómatas finitos y las expresiones regulares, al transformar un autómata en una expresión que representa el mismo lenguaje. Asimismo, la incorporación de un mecanismo de verificación permitió comparar los resultados de ambos métodos y comprobar su coincidencia en las cadenas evaluadas.

Por otra parte, la organización del código mediante paquetes y principios de programación orientada a objetos contribuyó a una estructura modular que facilita la comprensión y el mantenimiento del sistema. En conclusión, el proyecto permitió integrar los fundamentos teóricos de la asignatura con herramientas de desarrollo de software, fortaleciendo los conocimientos de programación y demostrando la utilidad de los autómatas en aplicaciones informáticas.

Bibliografía / egrafía

Bibliografía

- Hopcroft, J. E., Motwani, R., y Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3.ª edición. Pearson, 2006.
- Sipser, M. *Introduction to the Theory of Computation*. 3.ª edición. Cengage Learning, 2012.

Egrafía

- **GeeksforGeeks.** *Generating Regular Expression from Finite Automata.* Disponible en: https://www.geeksforgeeks.org/theory-of-computation/generating-regular-expression-from-finite-automata/
- **Oracle.** *Java Documentation*. Disponible en: https://docs.oracle.com/en/java/
- **Oracle.** *Class Pattern (Java API)*. Disponible en: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/regex/Pattern.html
- **Apache Maven.** *Maven Documentation*. Disponible en: https://maven.apache.org/guides/
- **JGraphX.** *mxGraph Documentation*. Disponible en: https://jgraph.github.io/mxgraph/

Las fuentes anteriores se consultaron como referencia para los fundamentos teóricos de los autómatas finitos, la conversión a expresiones regulares y las tecnologías utilizadas en el desarrollo del proyecto.

ANEXOS

En esta sección se presentan los diagramas UML detallados de los principales módulos que conforman el sistema Simulador de Autómatas. Estos diagramas complementan la arquitectura general presentada anteriormente y permiten observar con mayor detalle las clases, sus atributos, métodos y relaciones.

Con el propósito de facilitar su comprensión, la arquitectura del proyecto se dividió en cuatro diagramas, organizados según las responsabilidades de cada módulo.

Anexo A. Diagrama UML del modelo, controlador y simulación

Este diagrama representa las clases encargadas de almacenar la estructura matemática del autómata, administrar sus operaciones y ejecutar la simulación de cadenas.

Incluye las clases Automata, Estado, Transicion, AutomataController, SimuladorAFD, PasoSimulacion y ResultadoSimulacion.

Estas clases permiten gestionar los estados y las transiciones, procesar las cadenas ingresadas y determinar si son aceptadas o rechazadas.

Anexo B. Diagrama UML del algoritmo de conversión a Regex

Este diagrama presenta las clases responsables de transformar los autómatas finitos deterministas en expresiones regulares mediante el algoritmo de eliminación de estados.

Se incluyen las clases ConversorRegex, NormalizadorAutomata, ExpresionRegular, PasoConversion, ResultadoConversion y VerificadorEquivalencia, junto con las estructuras auxiliares utilizadas durante el proceso.

Su organización permite comprender cómo se realiza la normalización, la eliminación progresiva de estados y la obtención de la expresión regular resultante.

Anexo C. Diagrama UML de la interfaz gráfica y sus componentes

Este diagrama representa las clases responsables de la interacción entre el usuario y el sistema, incluyendo las ventanas principales, las herramientas de edición, la animación de simulaciones y la representación gráfica de los autómatas.

Se muestran las clases MainFrame, VentanaConversionRegex, PanelMensajes, EstilosUI, EditorVisualAFD, AnimadorSimulacion y los componentes del subpaquete graph.

Estas clases permiten construir una interfaz interactiva, visualizar los autómatas y presentar los resultados de las diferentes operaciones realizadas.

Anexo D. Diagrama UML del módulo de persistencia y restauración JSON

Este diagrama muestra las clases encargadas de guardar, importar y reconstruir los autómatas mediante archivos en formato JSON.

Incluye las clases AutomataJson, RestauradorModeloAFD y sus estructuras auxiliares, utilizadas para almacenar la información de los estados, las transiciones y sus posiciones.

Este módulo permite conservar los autómatas creados por el usuario y recuperarlos posteriormente para continuar su edición, simulación o conversión.

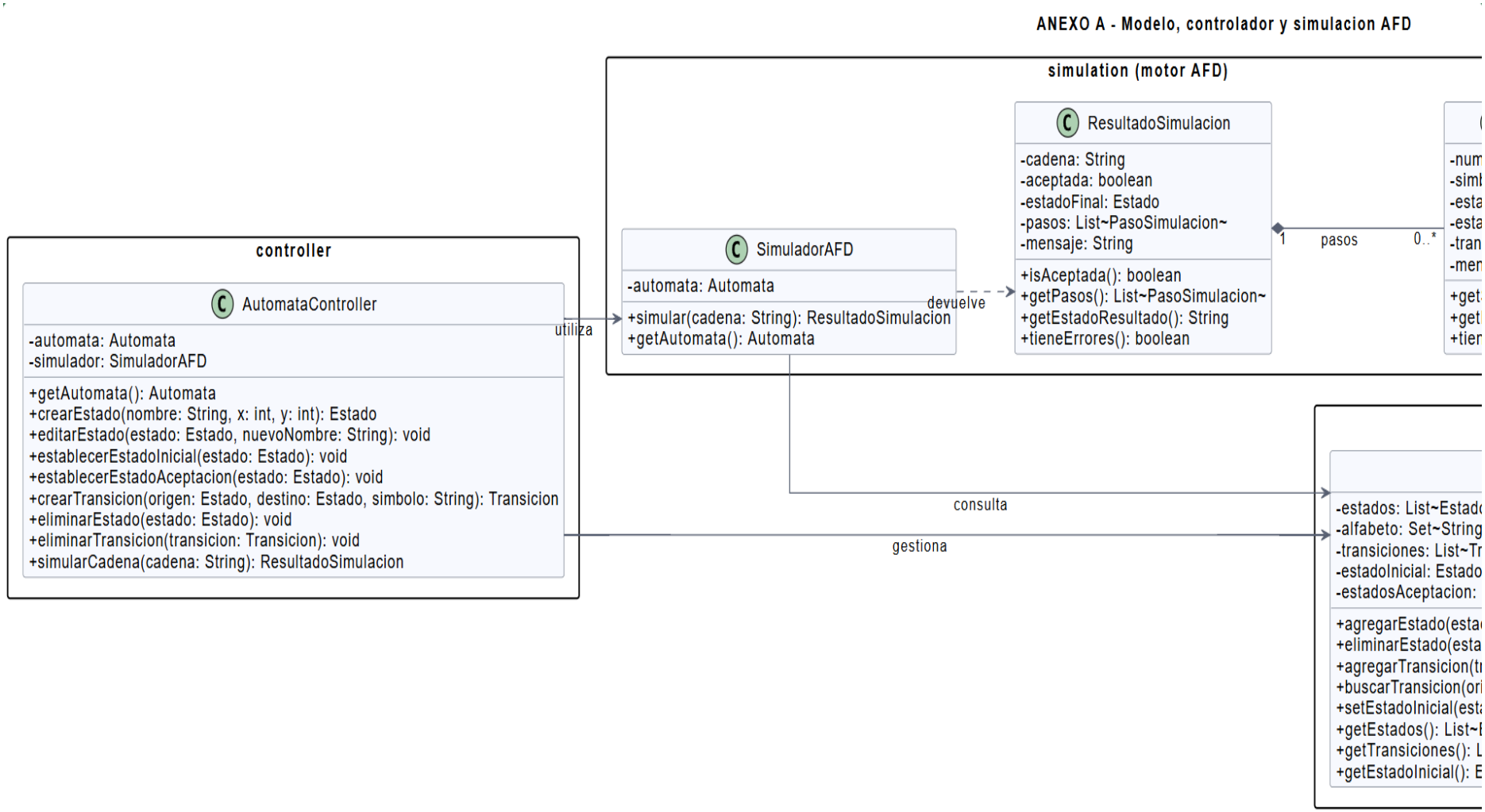


Figura1. Diagrama UML del modelo, controlador y simulación del AFD.

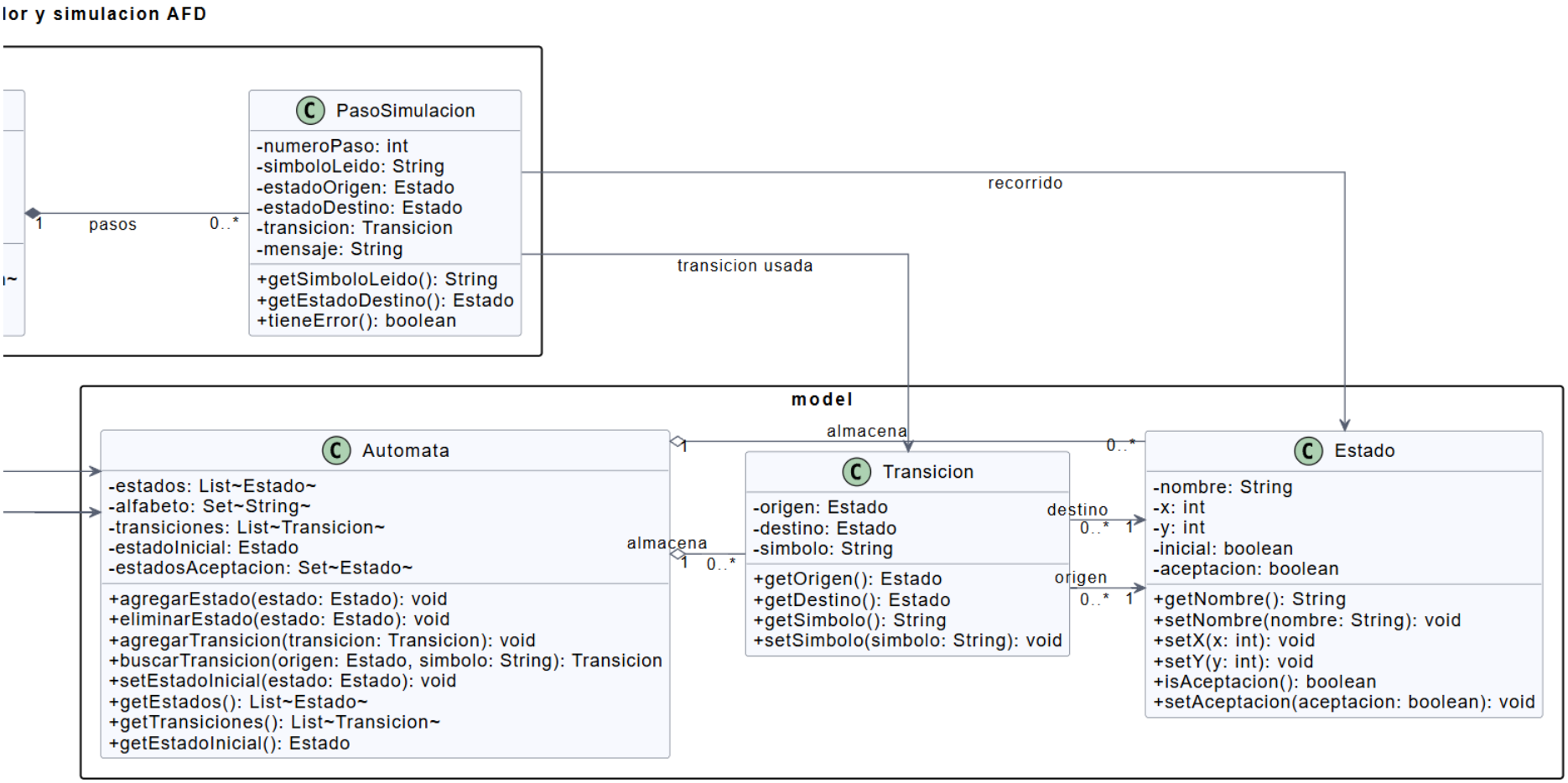


Figura2. Diagrama UML del modelo, controlador y simulación del AFD.

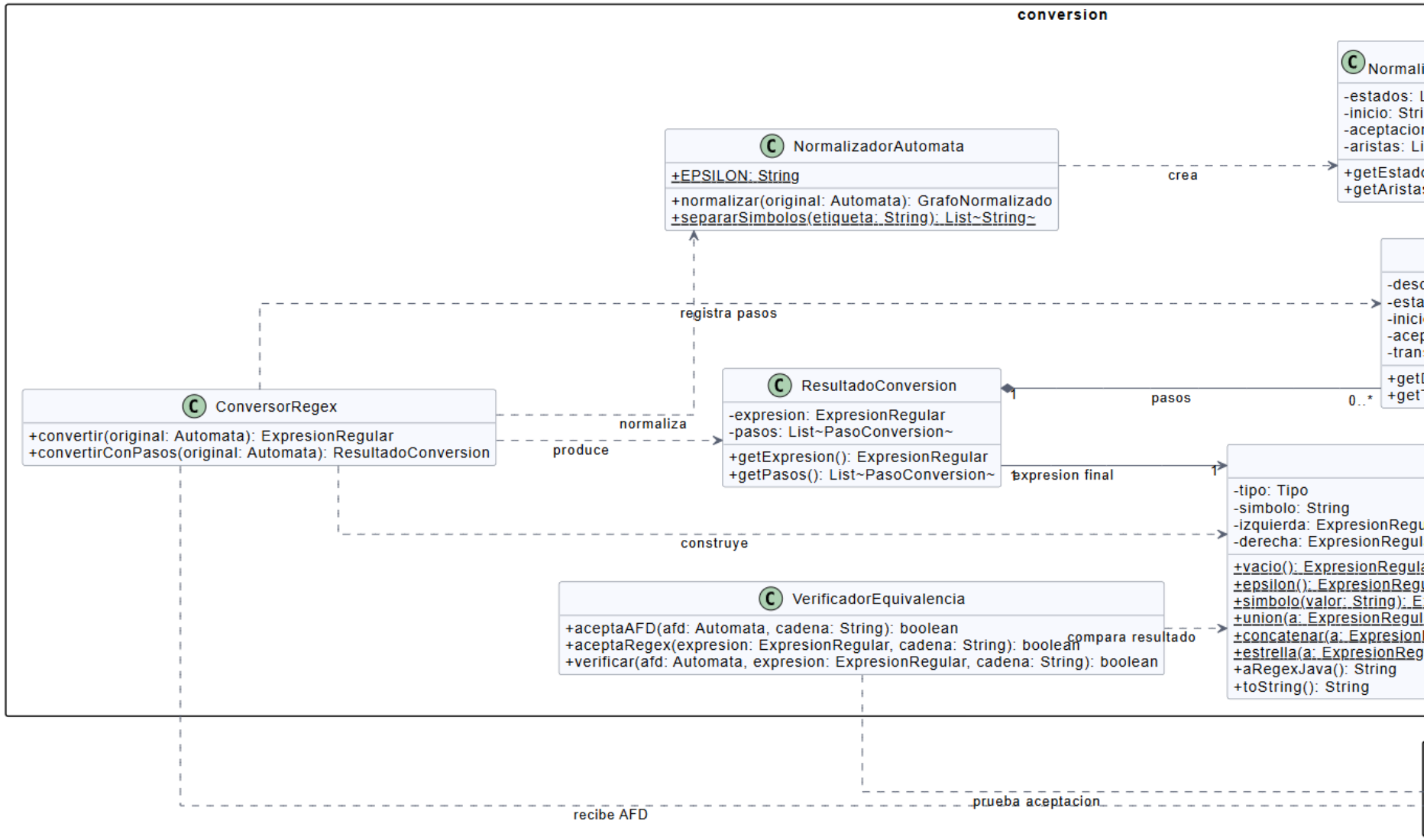


Figura1. Diagrama UML del módulo de conversión a expresiones regulares

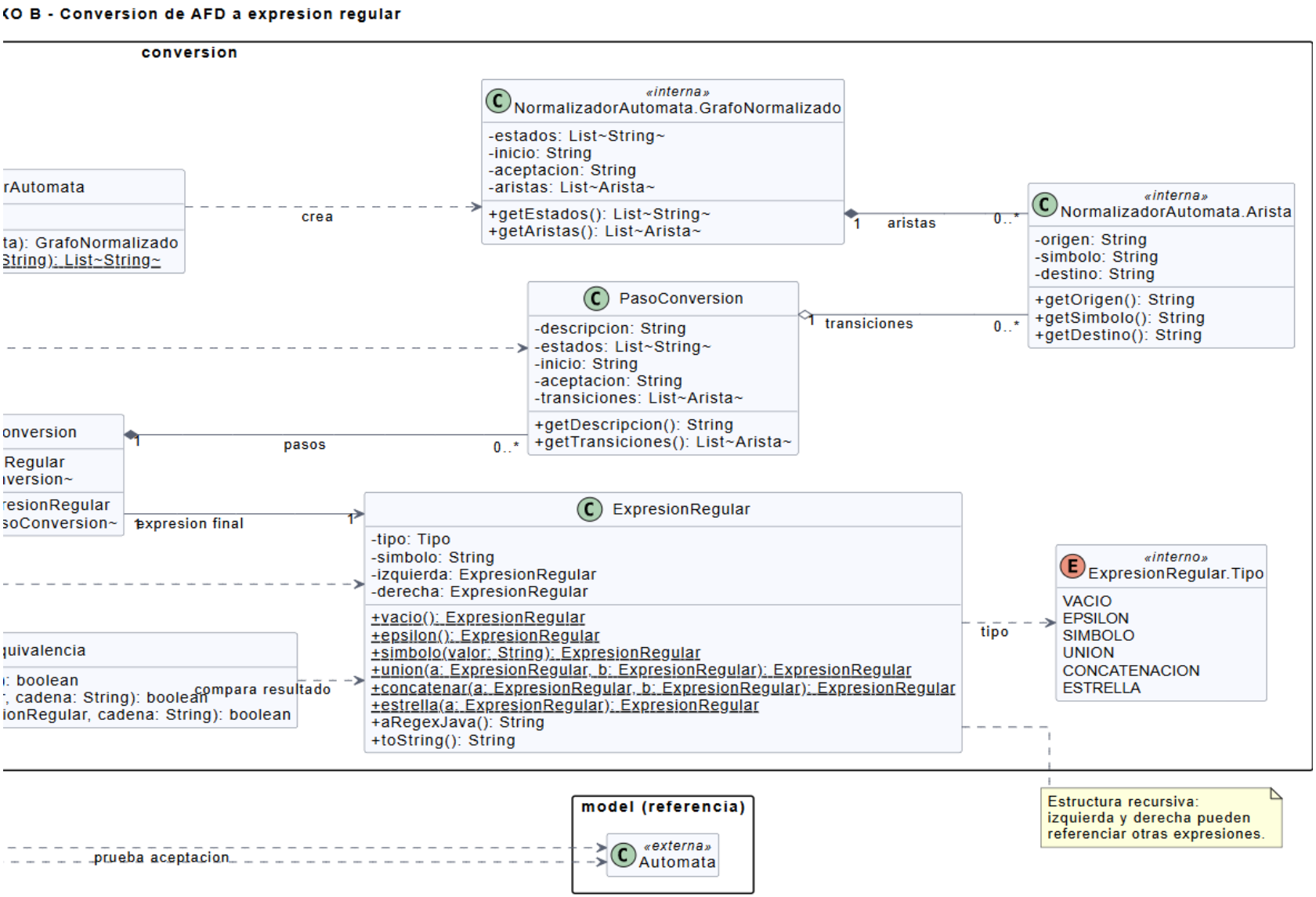


Figura1. Diagrama UML del módulo de conversión a expresiones regulares

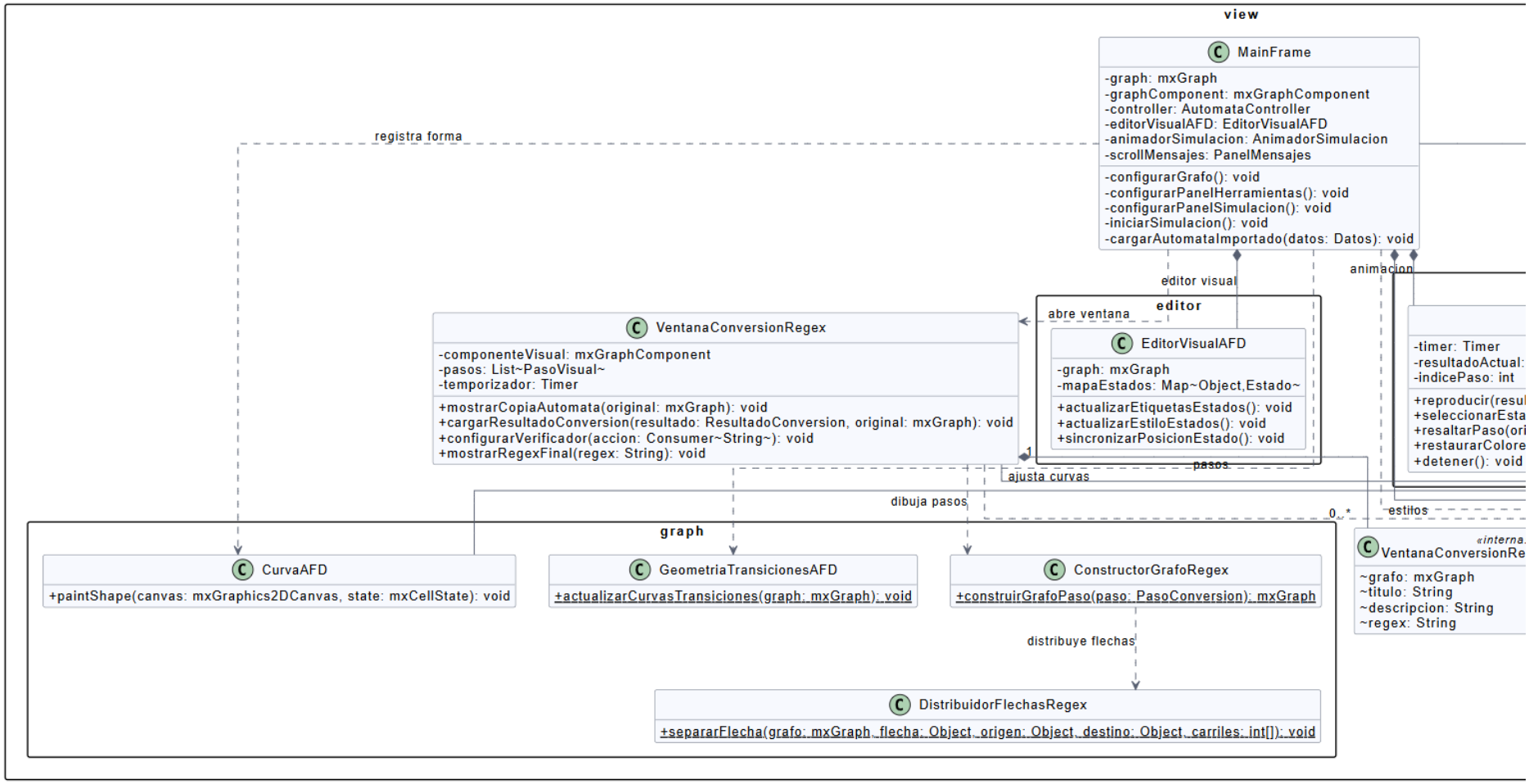


Figura1. Diagrama UML de la interfaz gráfica y sus componentes.

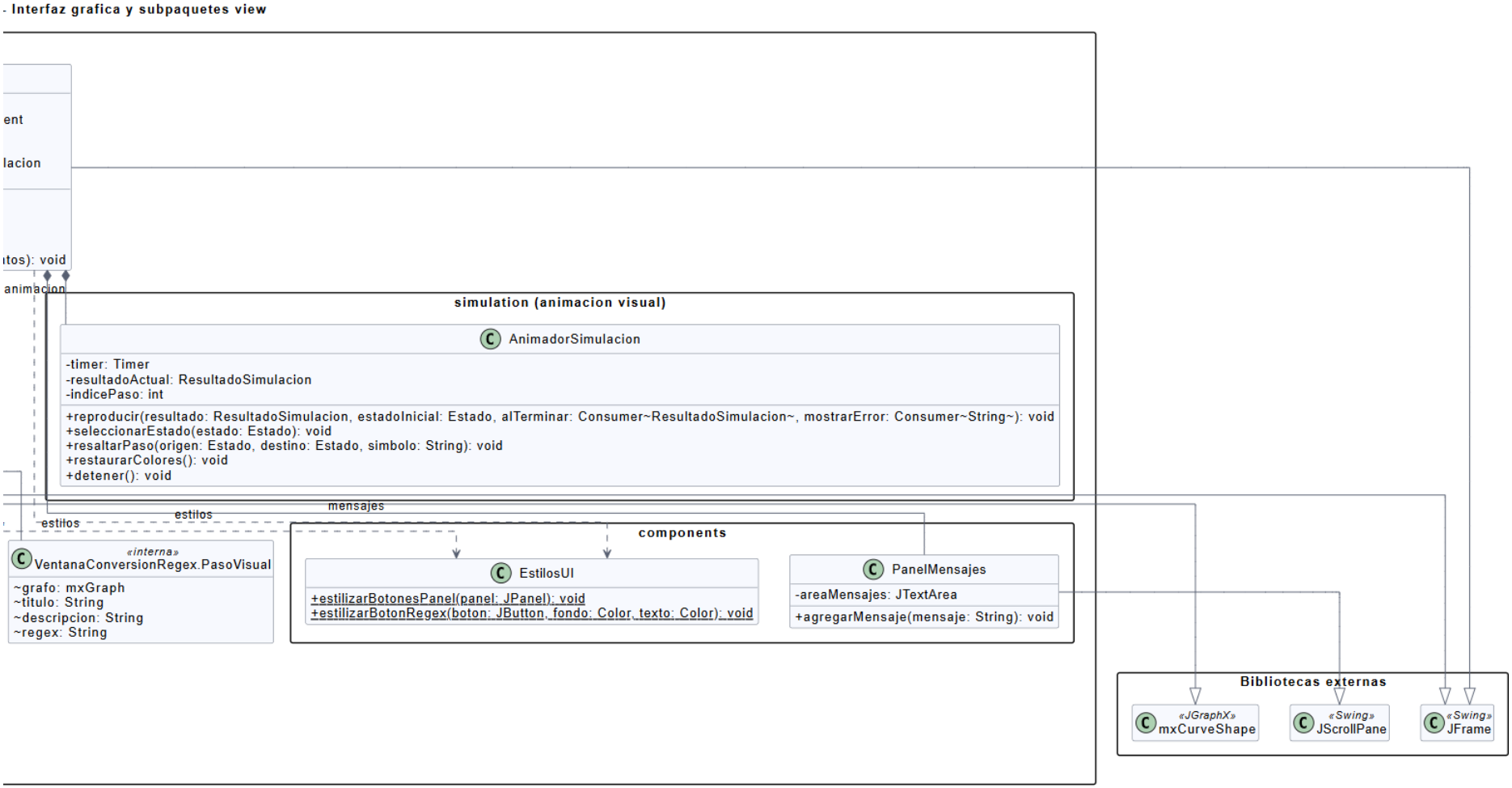


Figura2. Diagrama UML de la interfaz gráfica y sus componentes.

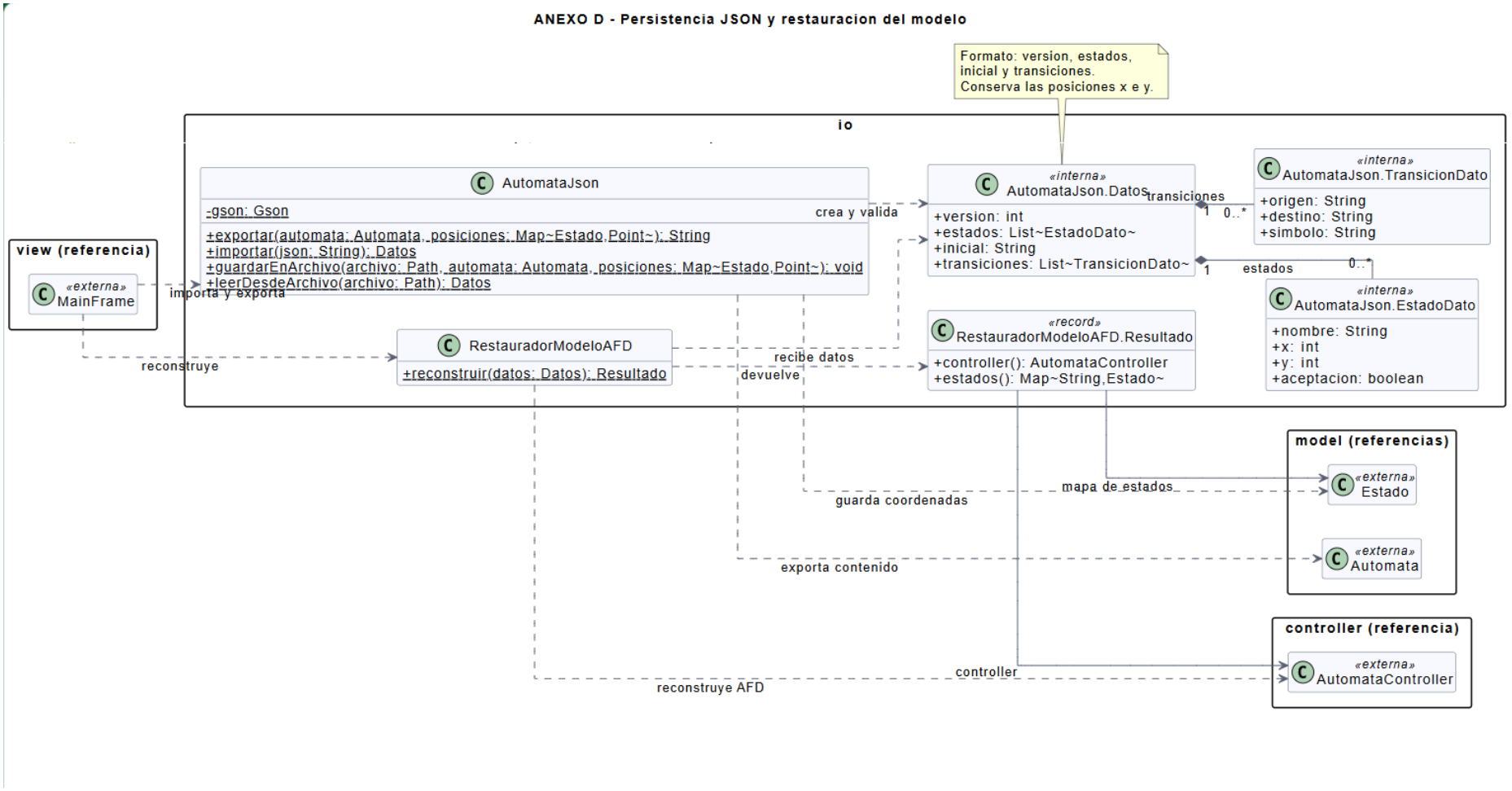


Figura. Diagrama UML del módulo de persistencia y restauración de autómatas.

Los diagramas presentados permiten complementar la descripción de la arquitectura del software, mostrando la organización interna del sistema y la separación de responsabilidades aplicada durante su implementación.